SPÉCIFICATION TECHNIQUE & FONCTIONNELLE

Migration WPF → ASP.NET MVC

Page principale — Tableau de données éditable en ligne avec DataTables.js

Document de référence pour l'équipe de développement

Version 1.0

Sommaire

1. Contexte et objectif
2. Périmètre fonctionnel
    1. Affichage du tableau
    2. Colonne « Customer » composite
    3. Édition en ligne (inline edit)
    4. Boutons de validation / annulation
    5. Bouton « Sauvegarder les changements »
3. Maquette fonctionnelle (wireframe)
4. Architecture technique proposée
5. Modèle de données / ViewModel
6. Implémentation DataTables (front-end)
    1. Structure HTML
    2. CSS — rendu de la colonne Customer
    3. Initialisation DataTables
    4. Passage en mode édition
    5. Gestion des boutons Valider / Annuler
    6. Sauvegarde globale
7. Implémentation ASP.NET MVC (back-end)
    1. Contrôleur
    2. Endpoints AJAX
8. Règles de gestion & cas limites
9. Checklist de recette

1. Contexte et objectif

L'application WPF existante expose, sur sa page principale, un tableau de données permettant à l'utilisateur de consulter des lignes, d'éditer certaines d'entre elles directement dans le tableau (édition « inline »), puis de valider ou d'annuler chaque modification ligne par ligne, avant d'enregistrer l'ensemble des changements en base de données via un bouton global.

L'objectif de ce document est de décrire précisément le comportement attendu afin de reproduire à l'identique cette expérience utilisateur dans la nouvelle application web **ASP.NET MVC**, en s'appuyant sur la librairie JavaScript **DataTables** pour l'affichage et l'interaction avec le tableau.

> **OBJECTIF CLÉ**
> Offrir une expérience d'édition en ligne équivalente à celle du WPF (édition directe dans la cellule, apparition dynamique des boutons de validation, sauvegarde différée/groupée), tout en restant dans les standards d'une application web MVC (AJAX, JSON, séparation contrôleur / vue).

2. Périmètre fonctionnel

2.1 Affichage du tableau

- Le tableau affiche la liste des enregistrements (ex. commandes / lignes clients) avec pagination, tri par colonne et recherche globale — fonctionnalités natives de DataTables.
- Chaque ligne représente un enregistrement identifié par une clé unique (`Id`).
- Certaines colonnes sont **éditables**, d'autres restent en **lecture seule** (à définir avec le métier / repris de la grille WPF existante).
- Une colonne « Actions » est réservée, en fin de ligne, à l'apparition conditionnelle des boutons Valider (✓) et Annuler (×).

2.2 Colonne « Customer » composite

La colonne **Customer** affiche **trois informations distinctes dans une seule cellule**, chacune avec un alignement horizontal spécifique à l'intérieur de la cellule :

Valeur	Alignement dans la cellule	Exemple
ClientId	Gauche (bord gauche de la cellule)	`10423`
Code client	Centre *(centré horizontalement dans la cellule)*	`CL-PARIS-01`
Currency	Droite (bord droit de la cellule)	`EUR`

Visuellement, dans une même ligne de cellule, on retrouve donc l'identifiant à gauche, le code client au centre, et la devise à droite — sans que ces trois informations ne soient dans des colonnes séparées.

Id	Customer			Montant	Date	Actions
1	10423	CL-PARIS-01	EUR	1 250,00	04/11/2026	

2	10987	CL-LYON-07	USD	980,50	05/11/2026	✓ ✕
3	10311	CL-NANTES-02	EUR	430,00	03/11/2026	

Figure 1 — Wireframe : la ligne 2 est en cours d'édition (fond jaune pâle), les boutons Valider / Annuler sont visibles uniquement sur cette ligne.

2.3 Édition en ligne (inline edit)

- Au clic (ou double-clic, à valider avec le métier) sur une cellule éditable, celle-ci se transforme en champ de saisie (`<input>` , `<select>` … selon le type de donnée).
- Plusieurs cellules de la même ligne peuvent être éditées avant validation (comme dans le WPF, qui autorise la modification de plusieurs champs avant d'enregistrer la ligne).
- Tant qu'aucune saisie n'a été effectuée sur la ligne, aucun bouton n'est visible.

2.4 Boutons de validation / annulation

RÈGLE DÉCLENCHANTE

Les boutons **✓ (Valider)** et **× (Annuler)** apparaissent dans la colonne Actions **dès qu'une saisie est détectée** (évènement `input` / `change`) sur un des champs éditables de la ligne — exactement comme dans l'application WPF.

Bouton	Icône	Comportement
Valider	✓ (vert)	Marque la ligne comme « modifiée » (état local, en mémoire côté client), fige/affiche les nouvelles valeurs, masque les boutons. *Aucun appel serveur immédiat* : l'enregistrement en base ne se fait qu'au clic sur le bouton global « Sauvegarder les changements » (cf. §2.5), pour reproduire le fonctionnement WPF où la validation ligne par ligne prépare la donnée avant un enregistrement groupé.
Annuler	× (rouge)	Restaure la/les valeur(s) d'origine de la ligne (avant édition), sort du mode édition, masque les boutons, retire la ligne de la liste des lignes modifiées si elle y figurait.

POINT À VALIDER AVEC LE MÉTIER

Dans le WPF, le bouton ✓ déclenche-t-il un enregistrement immédiat en base pour la ligne concernée, ou se contente-t-il de « confirmer » la saisie localement en attendant le bouton global ? Ce document part de la seconde hypothèse (cohérente avec la présence d'un bouton « Sauvegarder les changements » global), à confirmer avant développement.

2.5 Bouton « Sauvegarder les changements »

- Bouton unique, visible en permanence (ou activé uniquement si au moins une ligne a été validée localement — à définir).
- Au clic, l'ensemble des lignes marquées comme « modifiées » (validées via ✓) sont envoyées en un seul appel AJAX au contrôleur MVC, pour un enregistrement groupé en base de données.
- En cas de succès : notification de confirmation, réinitialisation de l'état « modifié » des lignes, rechargement ou mise à jour des données du tableau.
- En cas d'échec (partiel ou total) : affichage des erreurs, conservation des lignes en échec dans l'état « modifié » afin de ne pas perdre la saisie utilisateur.

3. Architecture technique proposée

Couche	Technologie	Rôle
Vue	Razor (.cshtml)	Rendu du tableau HTML, conteneur DataTables
Front-end	DataTables.js + jQuery + JS/CSS custom	Rendu, tri, pagination, édition inline, gestion des boutons
Échanges	AJAX (JSON)	Chargement des données, envoi des modifications
Contrôleur	ASP.NET MVC Controller	Exposition des actions `GetData` , `SaveChanges`
Accès données	Entity Framework / Dapper (selon existant)	Lecture / écriture base de données

4. Modèle de données / ViewModel

```
// Modèle exposé au tableau DataTables
public class CustomerRowViewModel
{
    public int Id { get; set; }

    // Composantes de la colonne "Customer"
    public int ClientId { get; set; }
    public string CustomerCode { get; set; }
    public string Currency { get; set; }

    public decimal Montant { get; set; }
    public DateTime Date { get; set; }

    // autres colonnes éditables reprises de l'existant WPF...
}

// DTO utilisé pour l'enregistrement groupé
public class SaveChangesRequest
{
    public List<CustomerRowViewModel> ModifiedRows { get; set; }
}
```

5. Implémentation DataTables (front-end)

5.1 Structure HTML

```
<table id="customersTable" class="table table-striped" style="width:100%">
  <thead>
    <tr>
      <th>Id</th>
      <th>Customer</th>
      <th>Montant</th>
      <th>Date</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody><!-- rempli par DataTables via ajax --></tbody>
</table>

<button id="btnSaveAll" class="btn btn-primary" disabled>
  Sauvegarder les changements
</button>
```

5.2 CSS — rendu de la colonne Customer

La cellule « Customer » utilise un conteneur en `position: relative` et trois éléments positionnés en `absolute` (gauche / centré / droite) afin de respecter l'alignement demandé à l'intérieur d'une seule et même cellule.

```
.customer-cell {
    position: relative;
    display: block;
    width: 100%;
    min-height: 20px;
}
.customer-cell .client-id {
    position: absolute;
    left: 0;
    text-align: left;
}
.customer-cell .customer-code {
    position: absolute;
    left: 50%;
    transform: translateX(-50%);
    text-align: center;
}
.customer-cell .currency {
    position: absolute;
    right: 0;
    text-align: right;
}
```

5.3 Initialisation DataTables

```
const table = $('#customersTable').DataTable({
    ajax: {
        url: '/Customers/GetData',
        dataSrc: ''
    },
    columns: [
        { data: 'id' },
        {
            data: null,
            render: function (row) {
                return `
                    <span class="customer-cell" data-id="${row.id}">
                        <span class="client-id">${row.clientId}</span>
                        <span class="customer-code">${row.customerCode}</span>
                        <span class="currency">${row.currency}</span>
                    </span>`;
            }
        },
        {
            data: 'montant',
            className: 'editable-cell',
            render: function (data) {
                return `<span class="cell-value">${data}</span>
                        <input type="text" class="cell-input d-none" value="${data}">`;
            }
        },
        { data: 'date' },
        {
            data: null,
            orderable: false,
            className: 'actions-cell',
            render: function () {
                return `
                    <span class="btn-validate d-none" title="Valider">✓</span>
                    <span class="btn-cancel d-none" title="Annuler">x</span>`;
            }
        }
    ]
});
```

5.4 Passage en mode édition

```
$('#customersTable tbody').on('click', 'td.editable-cell', function () {
    const $cell = $(this);
    $cell.find('.cell-value').addClass('d-none');
    $cell.find('.cell-input').removeClass('d-none').trigger('focus');
});
```

5.5 Gestion des boutons Valider / Annuler

Les boutons apparaissent **dès la première saisie** (évènement `input`) dans un champ éditable de la ligne, et stockent la ligne dans une liste des lignes modifiées (en mémoire, par `Id`) en attente de sauvegarde globale.

```
const modifiedRows = new Map(); // clé = Id de la ligne, valeur = données modifiées

// Apparition des boutons dès la saisie
$('#customersTable tbody').on('input', '.cell-input', function () {
    const $row = $(this).closest('tr');
    $row.find('.btn-validate, .btn-cancel').removeClass('d-none');
    $row.addClass('row-editing');
});

// Clic sur Valider (✓)
$('#customersTable tbody').on('click', '.btn-validate', function () {
    const rowData = table.row($(this).closest('tr')).data();
    const $row = $(this).closest('tr');

    // lecture des valeurs saisies dans les inputs de la ligne
    const updated = collectRowValues($row, rowData);

    modifiedRows.set(updated.id, updated);

    // repasse les cellules en lecture seule avec les nouvelles valeurs affichées
    applyReadOnlyState($row, updated);

    $row.removeClass('row-editing').addClass('row-pending-save');
    $row.find('.btn-validate, .btn-cancel').addClass('d-none');

    $('#btnSaveAll').prop('disabled', modifiedRows.size === 0);
});

// Clic sur Annuler (✗)
$('#customersTable tbody').on('click', '.btn-cancel', function () {
    const $row = $(this).closest('tr');
    const rowData = table.row($row).data(); // valeurs d'origine

    applyReadOnlyState($row, rowData); // restaure l'affichage initial

    modifiedRows.delete(rowData.id);
    $row.removeClass('row-editing row-pending-save');
    $row.find('.btn-validate, .btn-cancel').addClass('d-none');

    $('#btnSaveAll').prop('disabled', modifiedRows.size === 0);
});
```

5.6 Sauvegarde globale

```
$('#btnSaveAll').on('click', function () {
    const payload = { modifiedRows: Array.from(modifiedRows.values()) };

    $.ajax({
        url: '/Customers/SaveChanges',
        method: 'POST',
        contentType: 'application/json',
        data: JSON.stringify(payload),
        success: function (response) {
            if (response.success) {
                modifiedRows.clear();
                $('#btnSaveAll').prop('disabled', true);
                table.ajax.reload(null, false); // recharge sans reset pagination
                showNotification('success', 'Modifications enregistrées.');
            } else {
                showNotification('error', response.errors);
            }
        },
        error: function () {
            showNotification('error', 'Erreur lors de la sauvegarde.');
        }
    });
});
```

6. Implémentation ASP.NET MVC (back-end)

6.1 Contrôleur

```
public class CustomersController : Controller
{
    private readonly ICustomerService _customerService;

    public CustomersController(ICustomerService customerService)
    {
        _customerService = customerService;
    }

    public ActionResult Index()
    {
        return View();
    }

    [HttpGet]
    public JsonResult GetData()
    {
        var data = _customerService.GetAll(); // List<CustomerRowViewModel>
        return Json(data, JsonRequestBehavior.AllowGet);
    }

    [HttpPost]
    public JsonResult SaveChanges(SaveChangesRequest request)
    {
        if (request?.ModifiedRows == null || !request.ModifiedRows.Any())
            return Json(new { success = false, errors = "Aucune ligne à enregistrer." });

        var result = _customerService.UpdateRows(request.ModifiedRows);

        return Json(new
        {
            success = result.IsSuccess,
            errors = result.Errors
        });
    }
}
```

6.2 Endpoints AJAX — résumé

Endpoint	Méthode	Entrée	Sortie
`/Customers/GetData`	GET	—	JSON — liste des `CustomerRowViewModel`
`/Customers/SaveChanges`	POST	JSON — `SaveChangesRequest` (liste des lignes modifiées)	JSON — `{ success, errors }`

7. Règles de gestion & cas limites

- **Plusieurs lignes en édition simultanée :** l'utilisateur doit pouvoir éditer plusieurs lignes avant de cliquer sur « Sauvegarder les changements » (comportement WPF à confirmer).
- **Navigation / changement de page DataTables pendant une édition :** définir si les modifications non validées (✓) sont perdues ou conservées en mémoire par Id de ligne.
- **Concurrence :** gérer le cas où une ligne a été modifiée par un autre utilisateur entre le chargement et la sauvegarde (ex. vérification d'un timestamp / version).
- **Validation des données :** appliquer, côté client (avant apparition du bouton ✓ actif) et côté serveur, les mêmes règles de validation que l'application WPF (formats, champs obligatoires, plages de valeurs).
- **Annulation globale :** prévoir si nécessaire un moyen d'annuler toutes les modifications en attente en une seule action.

8. Checklist de recette

#	Scénario	Résultat attendu
1	Chargement de la page	Tableau affiché, aucun bouton ✓/× visible
2	Clic dans une cellule éditable	La cellule devient un champ de saisie
3	Saisie d'un caractère dans un champ éditable	Les boutons ✓ et × apparaissent sur la ligne
4	Clic sur ×	Valeur d'origine restaurée, boutons disparaissent
5	Clic sur ✓	Ligne marquée « modifiée », boutons disparaissent, bouton global activé
6	Colonne Customer	ClientId à gauche, Code client centré, Currency à droite, dans la même cellule
7	Clic sur « Sauvegarder les changements »	Toutes les lignes validées sont enregistrées en base en un seul appel
8	Erreur serveur lors de la sauvegarde	Message d'erreur affiché, lignes concernées conservées en état « modifié »
9	Tri / pagination / recherche DataTables	Fonctionnels, sans régression sur l'état d'édition en cours